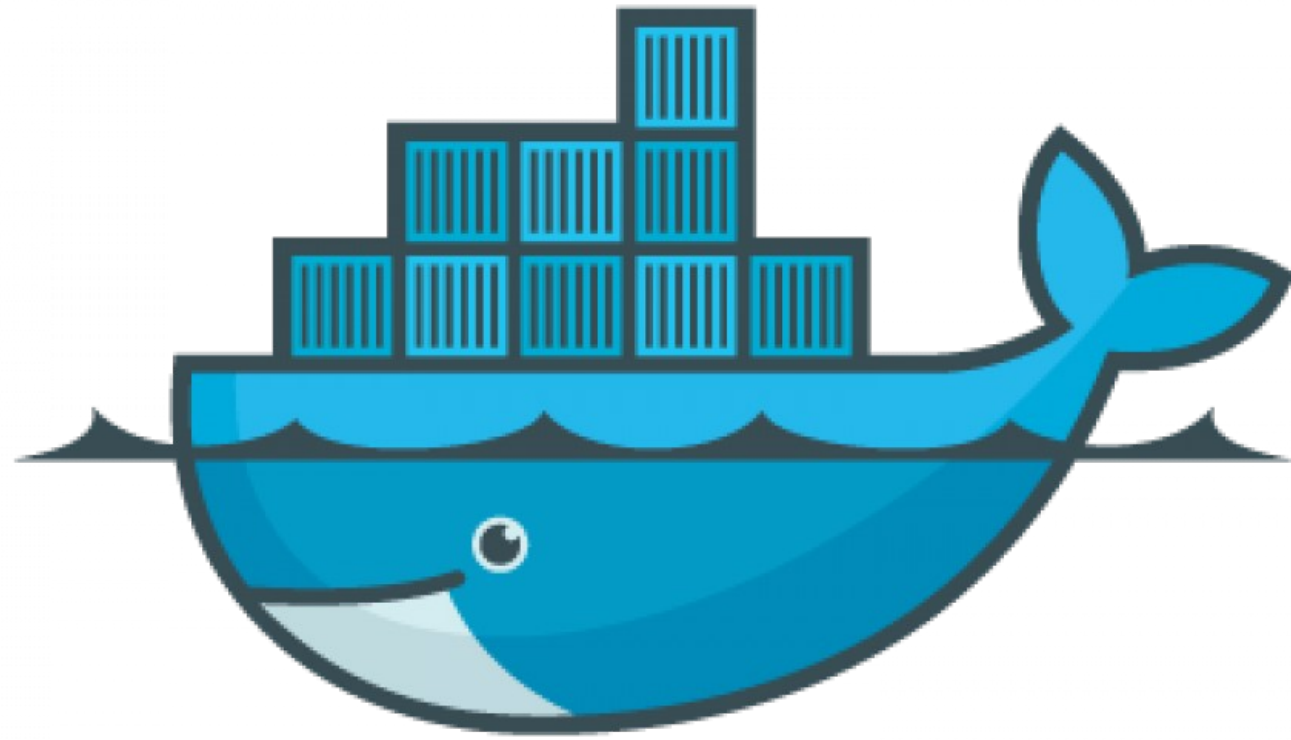
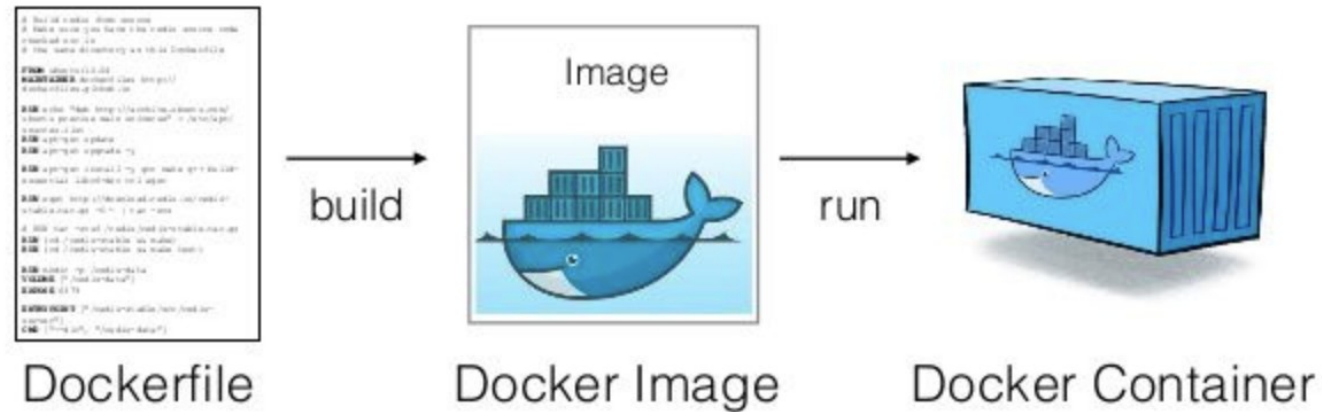




docker

이것만 알면 오늘 수업 끝 !



Dockerfile 을 이용해 DockerImage 를 만들고 DockerContainer 를 생성하고 시작한다 .

‘수학을 배워서 대체 어디에 쓰나요?’

포브스 선정
수학 선생님이 2000년 동안
제대로 대답 못한 질문 1위

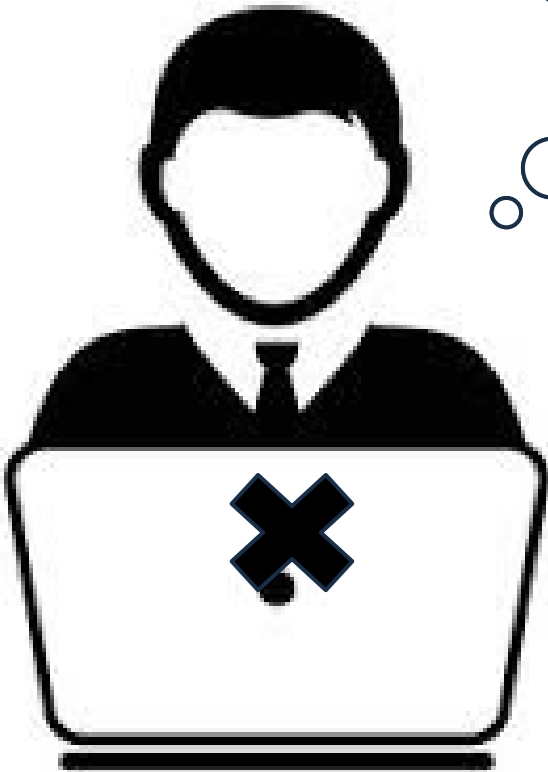


Forbes

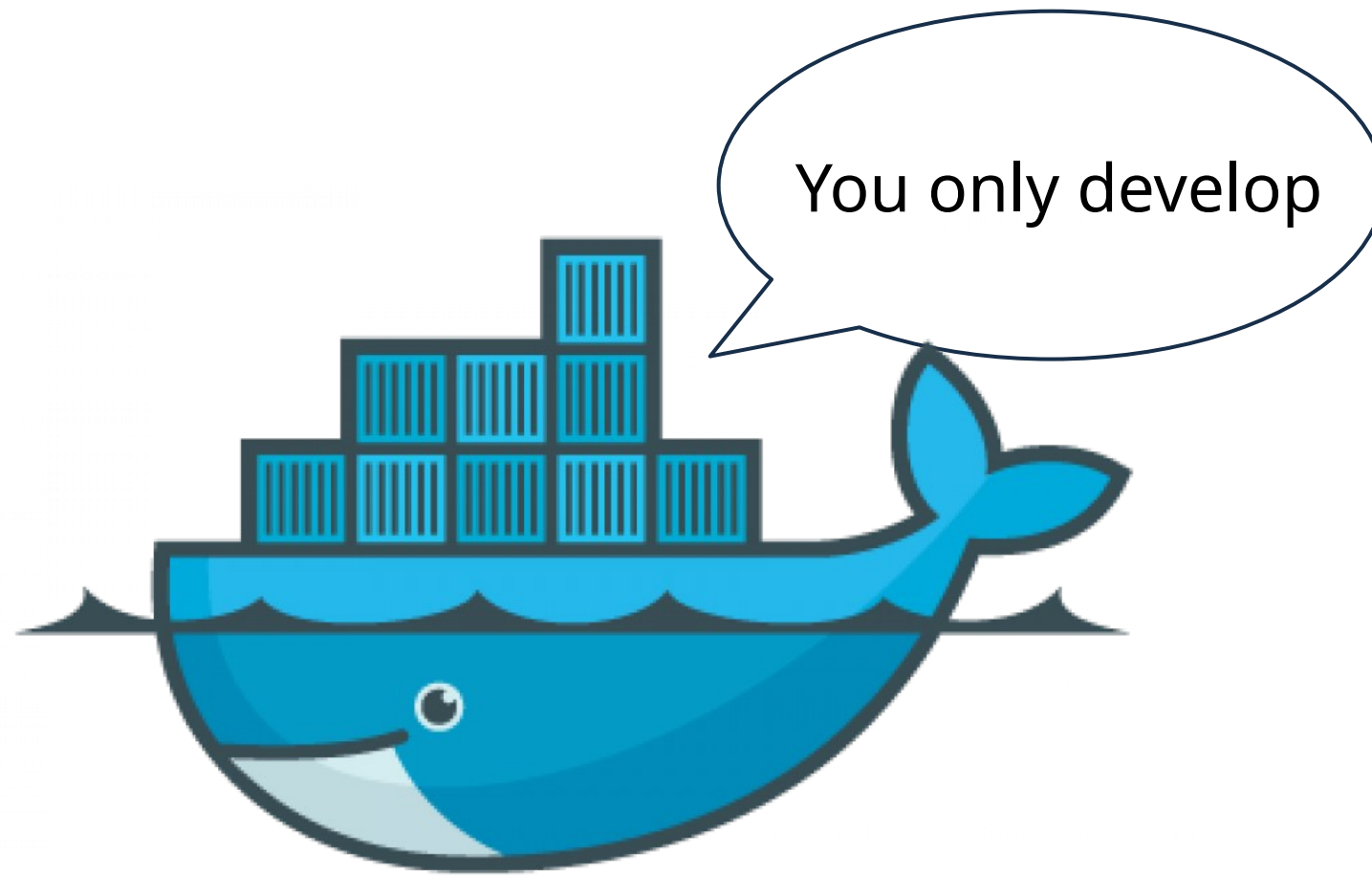
Why Docker?



Case. 1



Case. 1



docker

Case. 1

똑같은 환경

Case. 2



Hadoop
King!!



용량 / 메모리 / 부팅속도
6GB / 1GB / 20 초



Data_Node1.vdi

6GB / 1GB / 20 초

6GB / 1GB / 20 초



6GB / 1GB / 20 초



Data_Node1.vdi

6GB / 1GB / 20 초



용량 / 메모리 / 부팅속도
600GB / 100GB / 2000 초

Case. 2

BUY



Case. 2

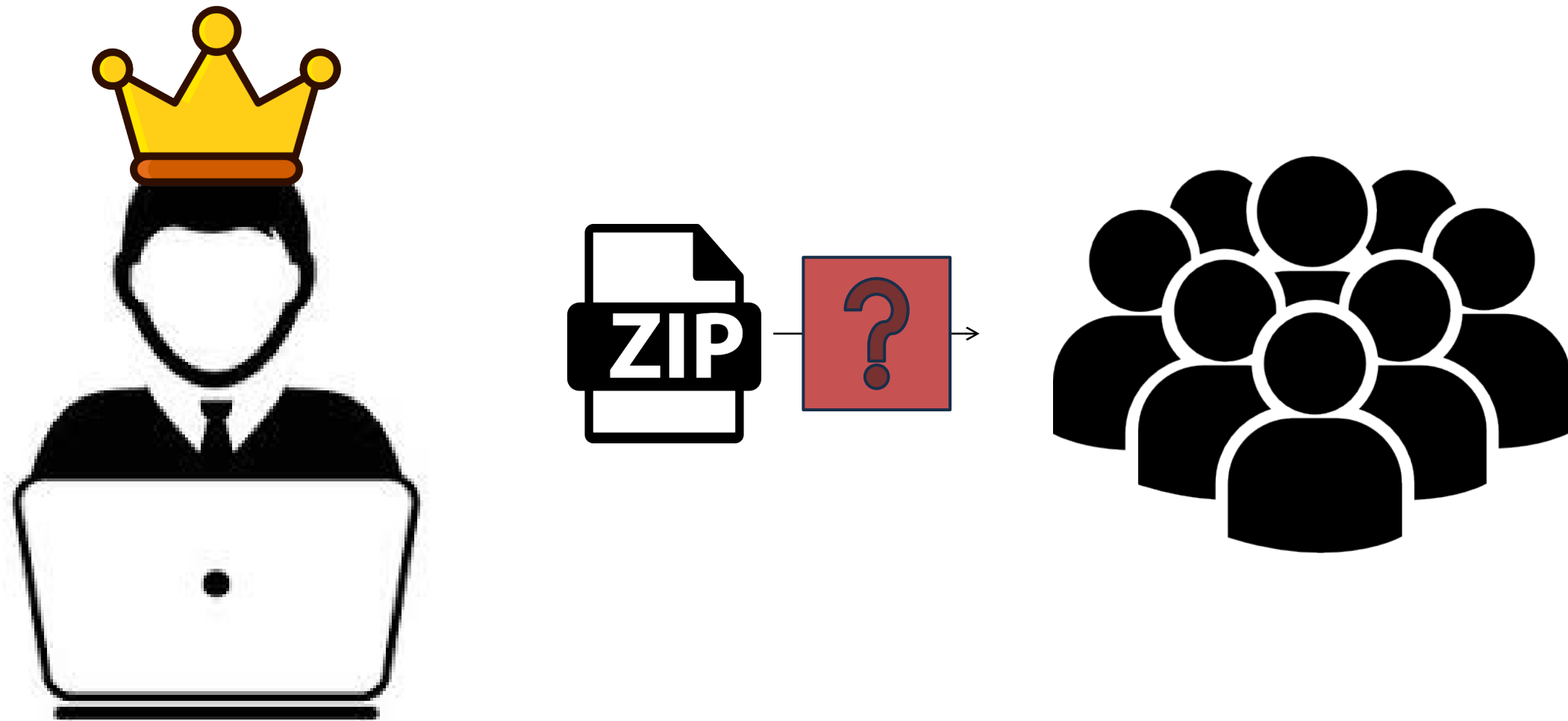


docker

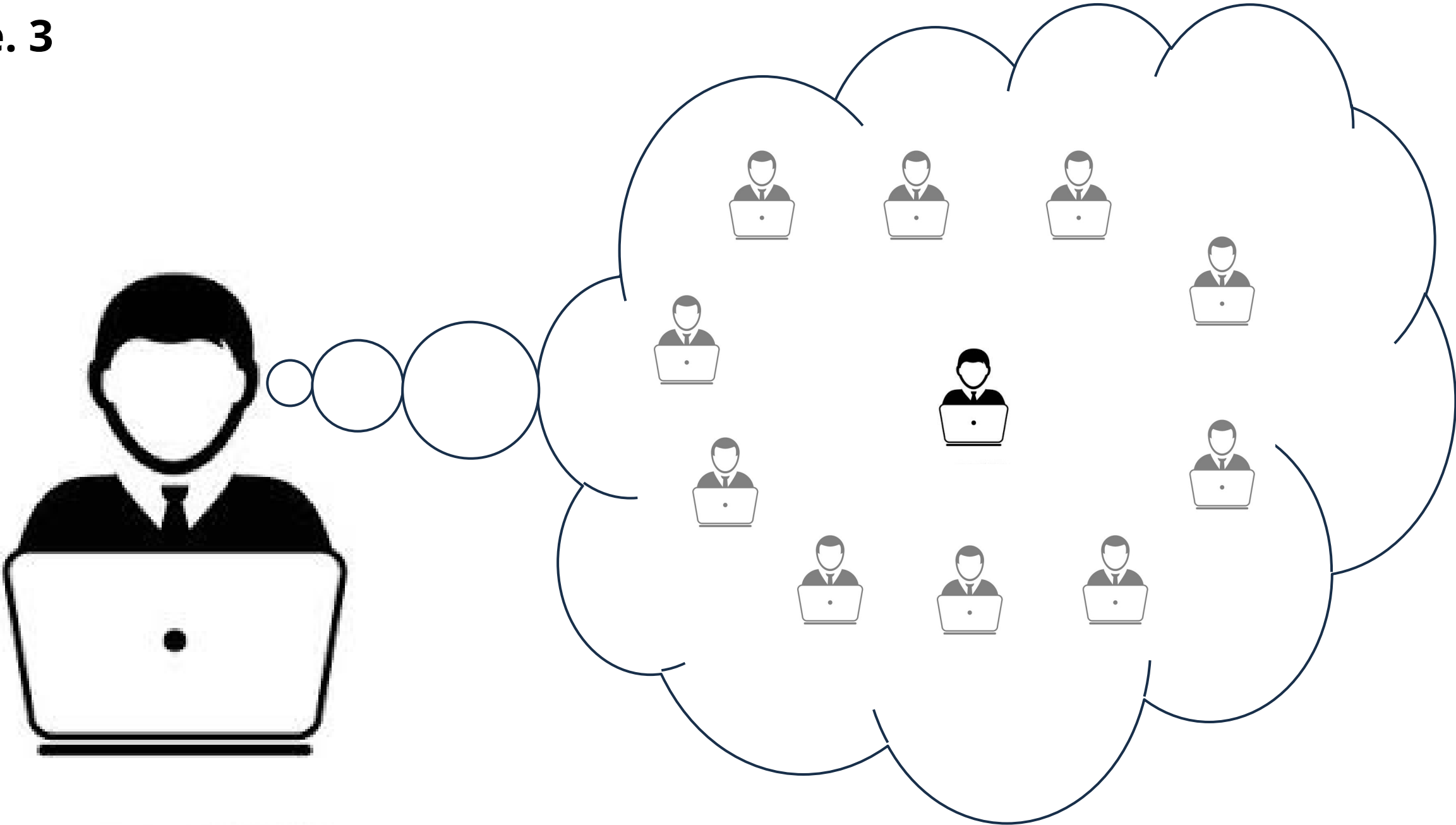
Case. 2

비용절감

Case. 3



Case. 3



Case. 3



docker

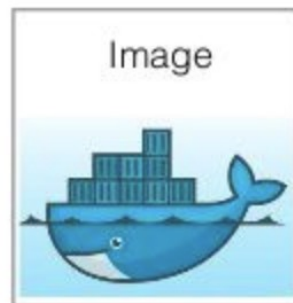
```
FROM ubuntu:16.04
MAINTAINER John Doe <john.doe@example.com>

RUN apt-get update \
    && apt-get install -y python3 \
    && python3 --help

RUN echo "Hello World" > /usr/local/bin/hello.py
```

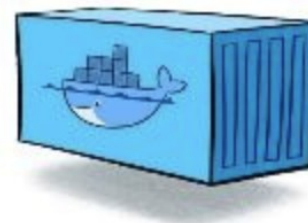
Dockerfile

build



Docker Image

run



Docker Container

← → ↕ ↑ > 내 PC > 로컬 디스크 (C:)

이름

\$WinREAgent
.CDN
20220302_lab
Apache Software Foundation
chrome
data
dice_docker
Doit_R-master
eclipse
hadoop_prj3
HadoopPrj
HashiCorp
Intel
javajungsuk_basic-master
javapri
jdk_1.8
LinPrj
OneDriveTemp
PerfLogs
Program Files
Program Files (x86)
ProgramData
RansomDefenderBackup
recovery

★ 즐겨찾기

바탕 화면
다운로드
문서
사진
LinPrj
02.python
Garage
Work
workspace

내 PC

3D 개체
다운로드
동영상
문서
바탕 화면
사진
음악
로컬 디스크 (C:)

네트워크

38개 항목 | 1개 항목 선택함

hadoop_prj3 속성

일반

공유

보안

이전 버전

사용자 지정

hadoop_prj3

| > | > | datanode

파일

홈

공유

보기

← → ↕ ↑ > 내 PC > 로컬 디스크 (C:) > hadoop_prj3 > docker-hadoop-spark-master > datanode

이름

Dockerfile
run.sh

수정한 날짜

유형

크기

2022-02-28 오전 12:24

파일

1KB

2022-02-28 오전 12:24

Shell Script

1KB

확인

취소

적용(A)

Case. 3

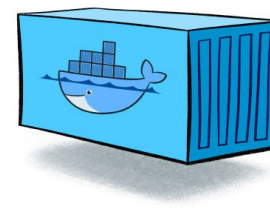
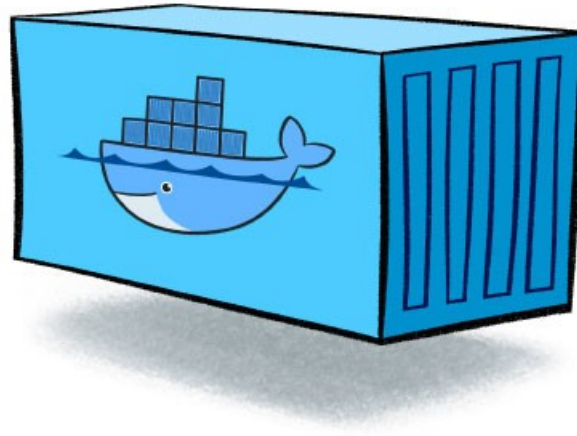
배포 개종음

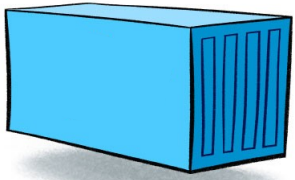
- 똑같은 환경
- 비용절감
- 배포 개종음



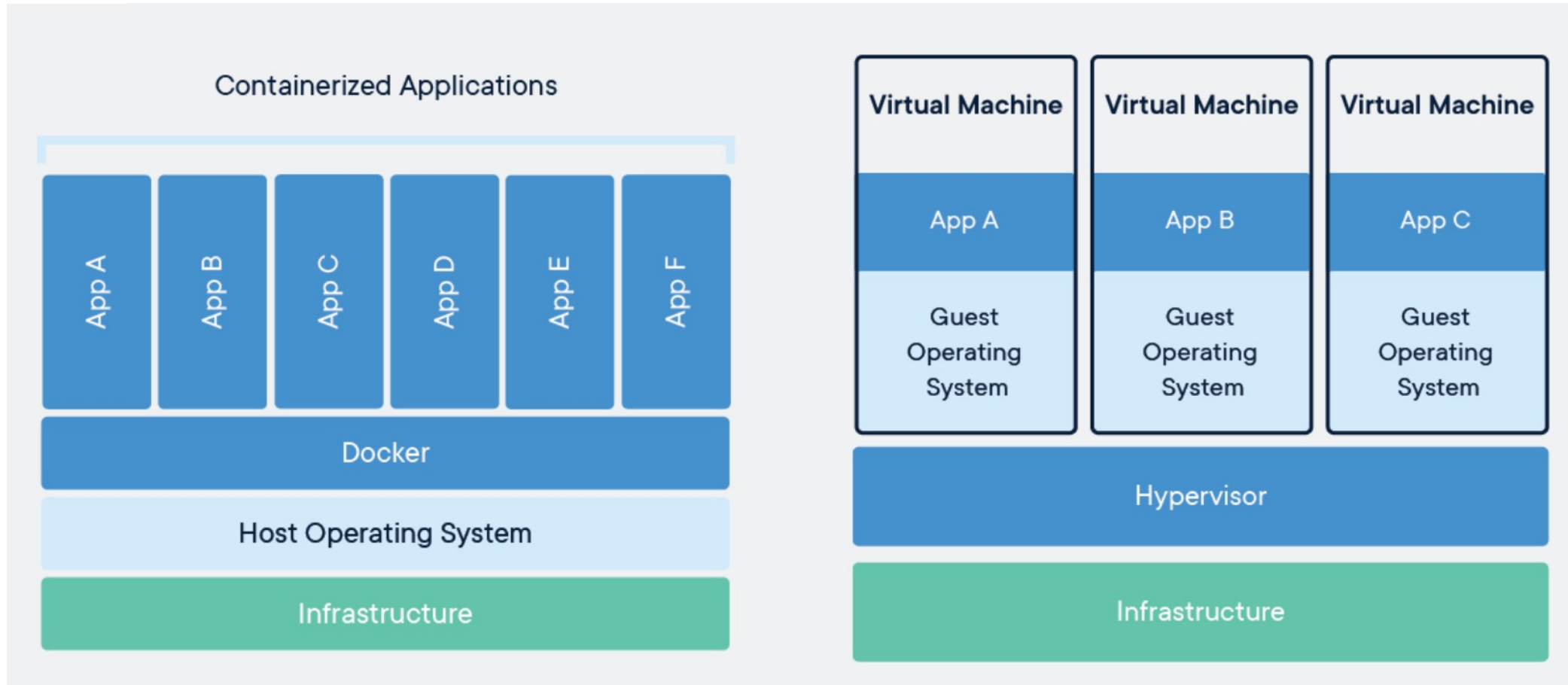
허리디스크에 굉장히 안 좋은 영향을 주게 되지.

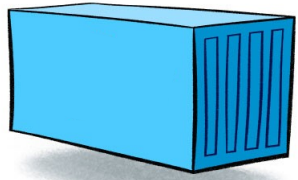
What is docker container?





1. 컨테이너는 VM 이 아니다 !





2. 컨테이너는 독립적이다 !

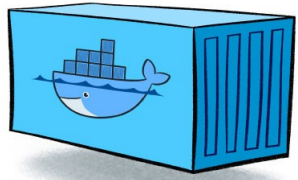
A



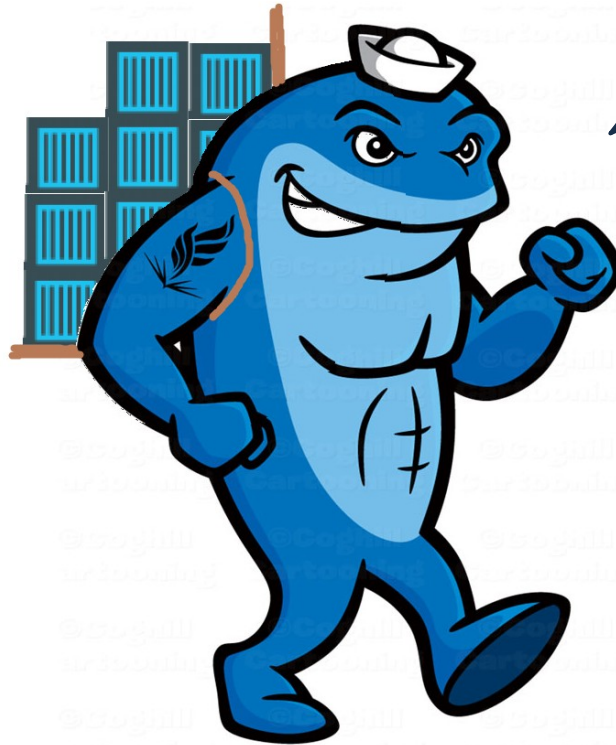
B



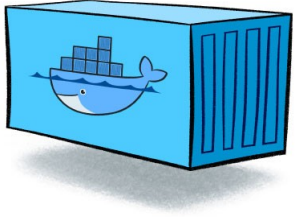
Linux kernel



3. 도커 컨테이너는 조작이 쉽다 .



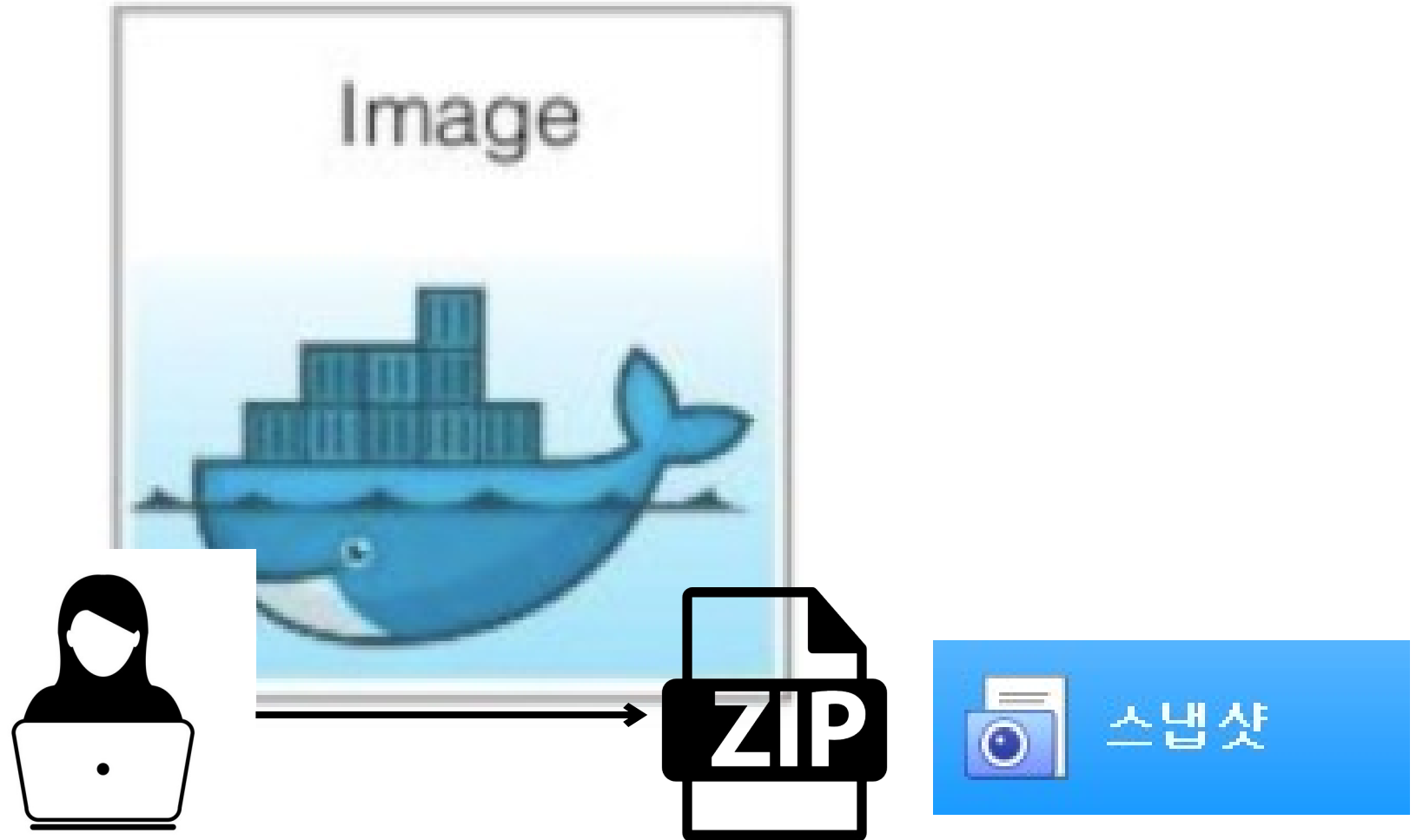
EZ

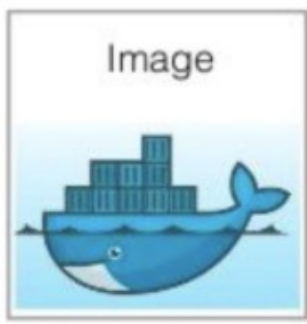


도커 컨테이너는요

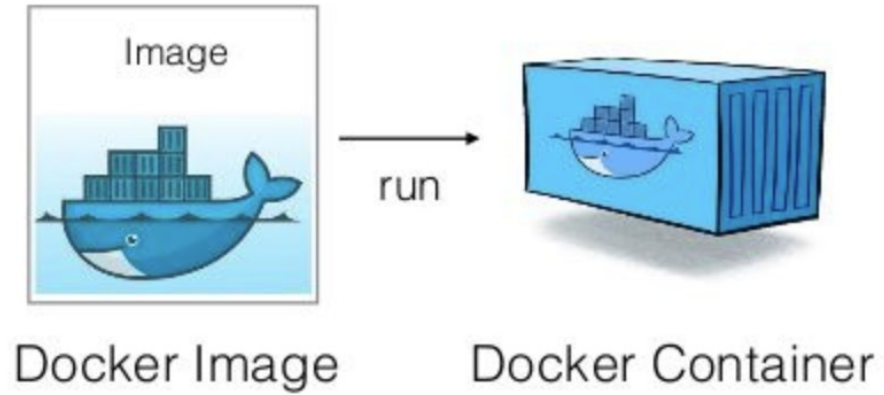
1. VM 이랑 다른겁니다
2. 완벽히 분리되어 있습니다 .
3. 사용이 쉽습니다 .

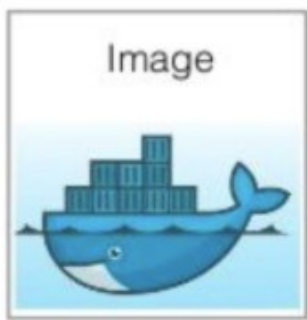
What is docker image?





1. 이미지는 컨테이너를 생성 , 구동할 때 사용된다 .





2. 이미지는 컨테이너로도 도커파일로도 만들어진다 .

```
FROM ubuntu:14.04
MAINTAINER John Doe <john.doe@example.com>

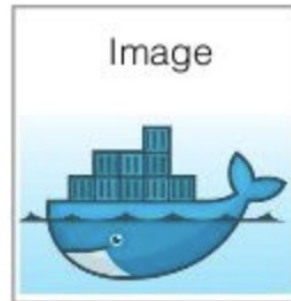
RUN apt-get update && apt-get install -y python
RUN apt-get install -y python-dev python-pip
RUN pip install Flask==0.10.1

EXPOSE 5000

CMD ["python", "app.py"]
```

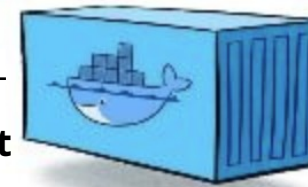
Dockerfile

build

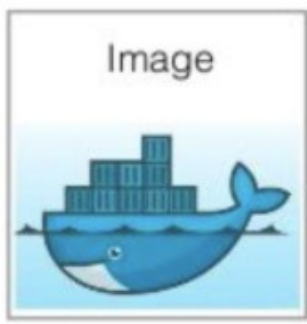


Docker Image

commit



Docker Container



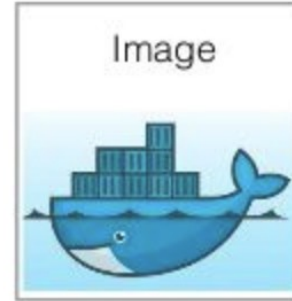
2. 이미지는 컨테이너로도 도커파일로도 만들어진다 .

1. 파이썬 3.6 버전
2. 환경 개빠르게
3. print(안성은 바보)

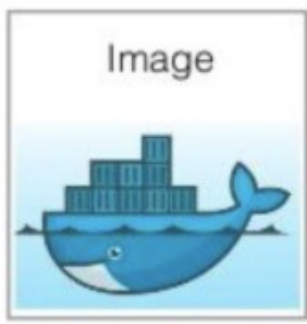
```
FROM python:3.6.0-slim
MAINTAINER anseon "안성은 바보"
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "main.py"]
```

Dockerfile

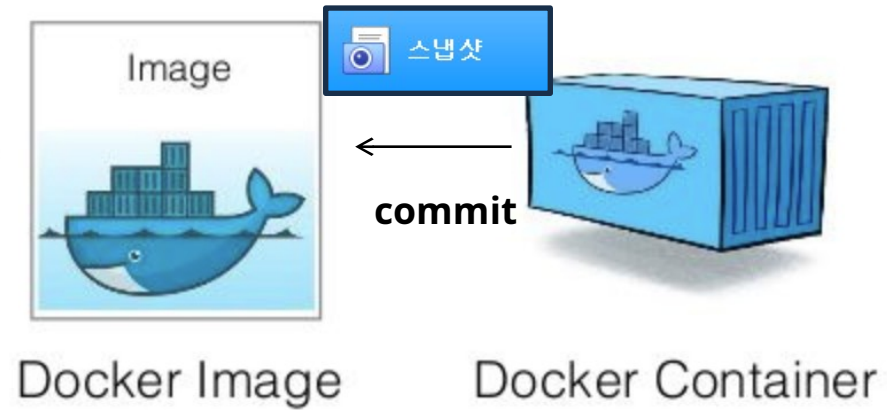
build

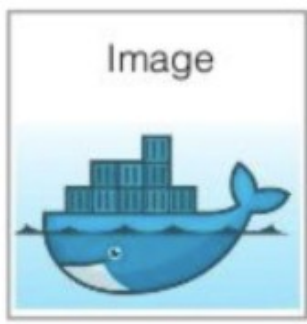


Docker Image

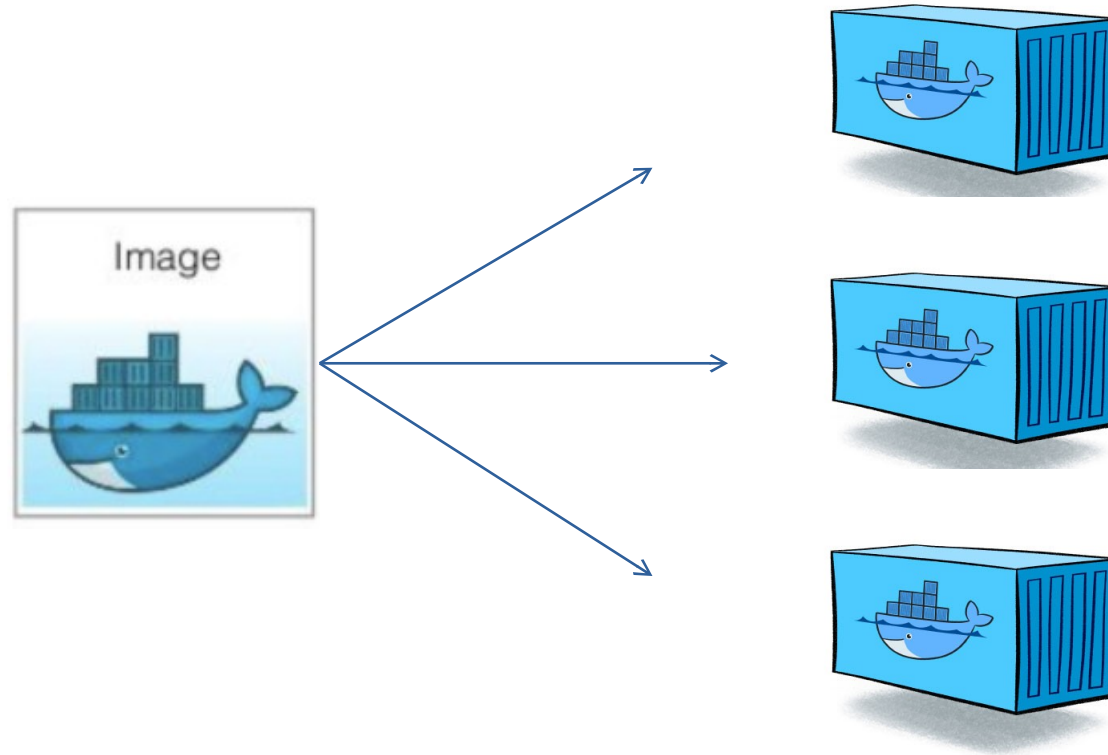


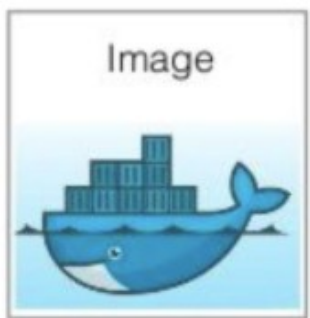
2. 이미지는 컨테이너로도 도커파일로도 만들어진다 .



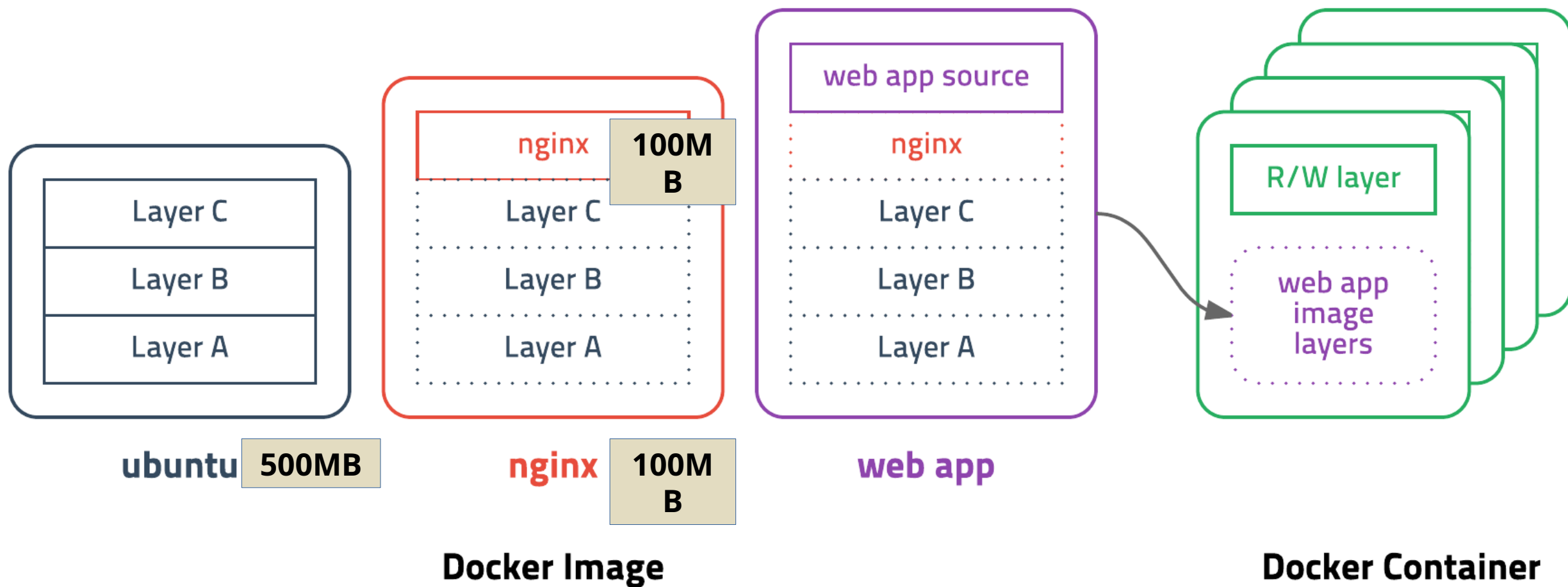


**3. 하나의 이미지는 여러 컨테이너를 생성할 수 있다 .
(컨테이너가 삭제되어도 이미지는 삭제되지 않는다)**





★ 이미지는 레이어 개념을 사용하고 있다.(겹치는 부분 공유하려고)



Image



도커 이미지는요

1. 컨테이너를 생성 , 구동할 때 쓰입니다 .
 2. 도커파일로도 컨테이너로도 만들 수 있습니다 .
(vagrantfile) (snapshot)
 3. 하나의 이미지는 여러 컨테이너를 생성할 수 있습니다 .
(컨테이너가 삭제되어도 이미지는 삭제되지 않습니다)
- ★. 이미지는 레이어 개념을 사용했다 .

What is dockerfile?

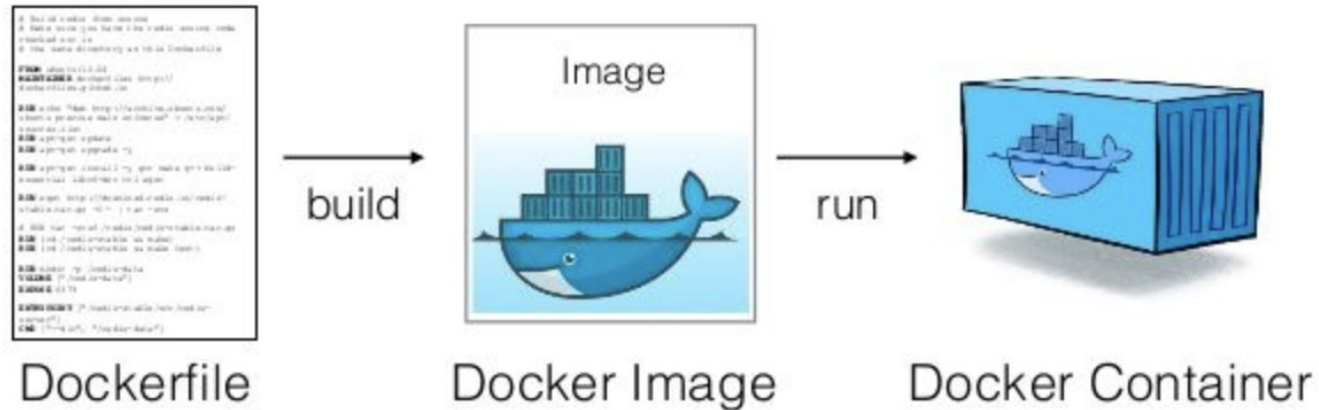
[illegible]

1. 도커 파일은 이미지 정보가 잘 담겨있다 .



도커를 쓰는 가장 큰 세가지 이유

1. 똑같은 환경
2. 비용절감
3. 배포 용이



도커 파일

1. 이미지 정보파악 쉬움

도커 이미지

1. 컨테이너 생성 / 구동
 2. 도커파일, 컨테이너로 생성가능
 3. 여러개 컨테이너 생성 가능
- ★. 레이어개념 사용함

도커 컨테이너

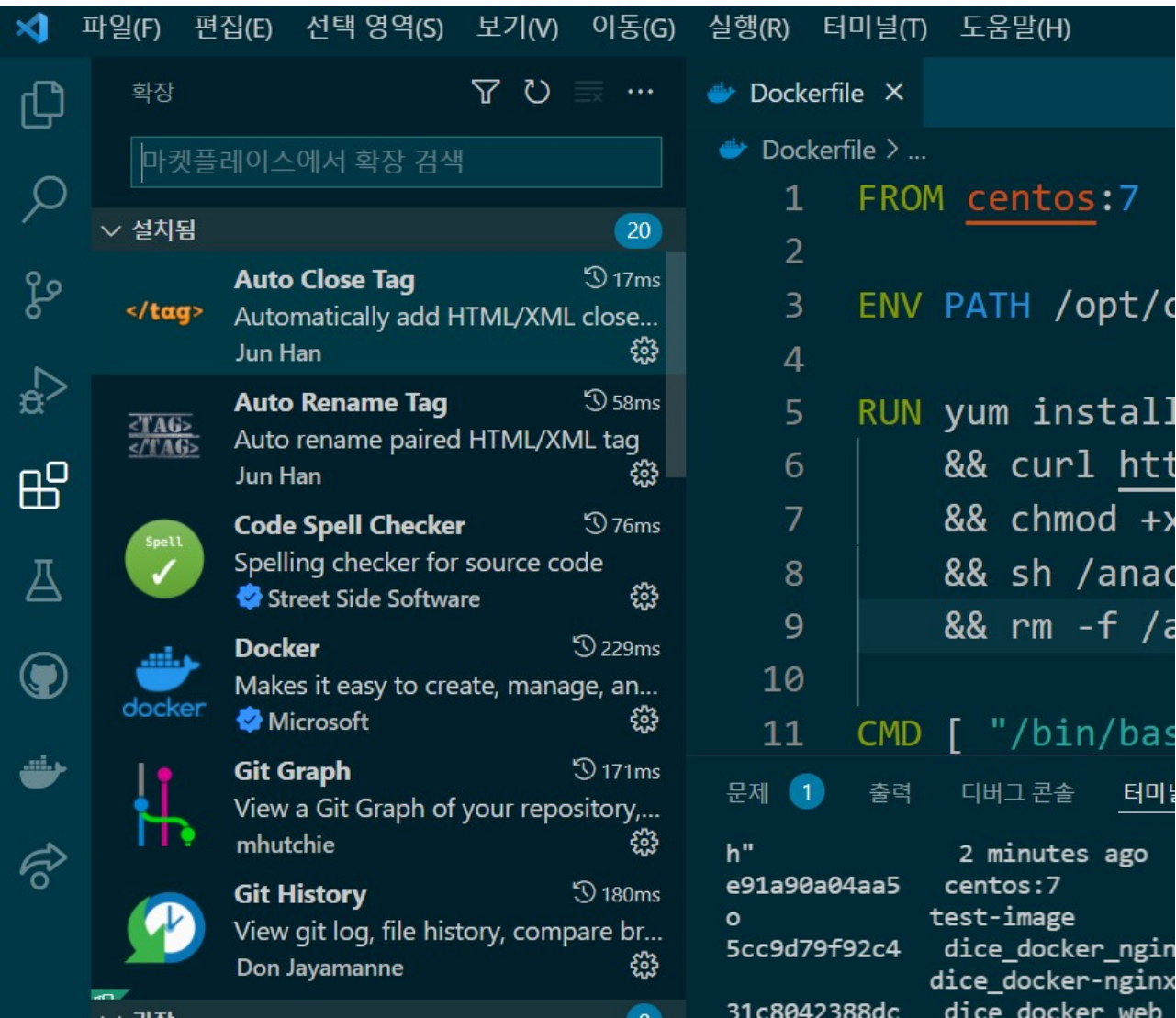
1. VM 이랑 다름
2. 완벽한 분리
3. 사용이 쉬움

]} 컨테이너 특징
— 도커의 특징 ★

실습 내용

1. 도커 파일로 도커 이미지 생성
2. 도커 컨테이너로 도커 이미지 생성 / 이미지로 컨테이너 생성
3. 도커허브에서 도커 이미지 받아와서 컨테이너 기동 (하둡)
(도커 이미지 제공자 : 갯정민)
4. 간단한 도커 명령어들

1. 도커 파일로 도커 이미지 생성 (아나콘다가 들어있는 간단한 도커파일)



1. vscode 열고 확장에서 Docker 설치

2. 아무거나 영문경로 폴더 열기
(파일 폴더 열기 -> 영문 폴더 아무거나)

3. 파일을 새로 하나 만들고 이름을
Dockerfile 로 지정

4. 다음 피피티 내용 자신의 컴퓨터에
맞게 복붙 후 저장

5. ctrl + shift + ~ 눌러서 cmd 터미널

6. docker build -t conda:0.1 .

7. docker image list

1. 도커 파일로 도커 이미지 생성 (아나콘다가 들어있는 간단한 도커파일)

Windows 사용자

```
FROM centos:7
```

```
ENV PATH /opt/conda/bin:$PATH
```

```
RUN yum install -y libXcomposite libXcursor libXi libXtst libXrandr alsa-lib mesa-libEGL libXdamage mesa-  
libGL libXScrnSaver python3 \  
  && curl https://repo.anaconda.com/archive/Anaconda3-2021.11-Linux-x86_64.sh -o /anaconda.sh \  
  && chmod +x /anaconda.sh \  
  && sh /anaconda.sh -b -p /opt/conda \  
  && rm -f /anaconda.sh
```

```
CMD [ "/bin/bash" ]
```

Mac M1 사용자

```
FROM --platform=linux/amd64 centos:7
```

```
ENV PATH /opt/conda/bin:$PATH
```

```
RUN yum install -y libXcomposite libXcursor libXi libXtst libXrandr alsa-lib mesa-libEGL libXdamage mesa-libGL libXScrnSaver \  
  && curl https://repo.anaconda.com/archive/Anaconda3-2021.11-Linux-x86_64.sh -o /anaconda.sh \  
  && chmod +x /anaconda.sh \  
  && sh /anaconda.sh -b -p /opt/conda \  
  && rm -f /anaconda.sh
```

```
CMD [ "/bin/bash" ]
```

2. 도커 컨테이너로 도커 이미지 생성 / 이미지로 컨테이너 생성

1. `docker run -it --name test-conda conda:0.1`

2. `docker ps -a`

3. `docker commit test-conda conda:0.2`

3. 하둡 파일 시스템 맛보기

1. `docker network create mdgome-hadoop_default`
2. `docker-compose up --build`
3. 인터넷에서 `localhost:9870` 해서 잘 되는지 확인

4. 도커 간단한 명령어 (설명)

docker start : 생성된 컨테이너를 실행함

docker create : 이미지를 기반으로 컨테이너 생성

docker run : docker create + docker start

docker exec : 백그라운드 실행 (컨테이너 접속 정보 죽어도 컨테이너 안죽음)★

docker attach : 포그라운드 실행 (컨테이너 접속 정보 죽으면 컨테이너 죽음)★

docker cp : 파일 또는 디렉토리 전체를 옮김

docker commit : 실행되는 컨테이너를 이미지로 만듦

docker build : 도커파일을 기반으로 이미지 생성

docker image : 도커 이미지 관련 각종 명령어

docker inspect : 도커 실행 후 컨테이너 정보

docker kill : 비정상 종료 (도커 종료가 안될때 강제종료)

docker logs : 컨테이너 내부 로그 보기

docker ps : 현재 도커 상황

docker pull, push : 도커 허브에서 이미지 가져올 때

docker stop : 정상종료

docker rm : 컨테이너 삭제

4. 도커 간단한 명령어 (표준)

docker start : docker start [이미지이름]:[태그]
docker create : docker create [옵션] [이미지이름]:[태그]
docker run : docker run [옵션] [이미지이름]:[태그]
docker exec : docker exec [옵션] CONTAINER COMMAND
docker attach : docker attach [이미지 이름]:[태그]
docker cp : docker cp [host 파일경로] [container name]:[container 내부 경로]
docker commit : docker commit [옵션] [컨테이너 이름] [저장소 [: 태그]]
docker build : docker build [옵션] [도커파일 경로]
docker image : docker image [명령어]
docker inspect : docker inspect [컨테이너 이름]
docker kill : docker kill [옵션] [컨테이너 이름]
docker logs : docker container logs -t [컨테이너식별자]
docker ps : docker ps [옵션]
docker pull : docker pull [도커허브아이디]/[이미지이름]:[태그]
docker stop : docker stop [컨테이너 이름]
docker rm : docker rmi [이미지 id] // docker rm [컨테이너 이름]

4. 도커 간단한 명령어 (예시)

docker start : docker start test-conda

docker create : docker create -it --name test-conda conda:0.1

docker run : docker run -it --name test-conda conda:0.1

docker exec : docker exec -it test-conda /bin/bash

docker attach : docker attach test-conda

docker commit : docker commit test-conda conda:0.2

docker build : docker build -t conda:0.1 .

docker image : docker image list

docker inspect : docker inspect test-conda

docker kill : docker kill test-conda

docker logs : docker logs test-conda

docker ps : docker ps -a

docker pull : docker pull python

docker stop : docker stop test-conda

docker rm : docker rmi conda:0.2 // docker rm test-conda

4. 도커 간단한 명령어 (옵션 , 많이쓰는거)

옵션	설명
-d	detached mode 흔히 말하는 백그라운드 모드(데몬 프로세스)
-p	호스트와 컨테이너의 포트를 연결 (포워딩)
-v	호스트와 컨테이너의 디렉토리를 연결 (마운트)
-e	컨테이너 내에서 사용할 환경변수 설정
--name	컨테이너 이름 설정
--it	-i와 -t를 동시에 사용한 것으로 터미널 입력을 위한 옵션 (컨테이너의 표준 입력과 로컬 컴퓨터의 키보드 입력을 연결)
--rm	프로세스 종료시 컨테이너 자동 제거
--link	컨테이너 연결 [컨테이너 명:별칭]
--restart	docker desktop을 실행시킬때마다 container의 자동 restart 실행 여부 - no: container를 재시작 시키지 않는다. (default) - on-failure[:max-retries]: container가 정상적으로 종료되지 않은 경우(exit code가 0이 아님)에만 재시작 시킨다. max-retries도 함께 주면 재시작 최대 시도횟수를 지정할 수 있고, 테스트 서버 등과 같은 리모트에 설정하면 좋을 것 같다. - always: container를 항상 재시작시킨다. exit code 상관 없이 항상 재시작 된다. - unless-stopped: container를 stop시키기 전 까지 항상 재시작 시킨다.